Universidad de Las Palmas de Gran Canaria

Grado en Ciencia e Ingeniería de Datos

Big Data - Academic Year 2026/2027

Optimised Matrix Multiplication and Sparse Matrices

Individual Assignment 2

Sara Carrizosa Tomé

GitHub Repository

`https://github.com/saracarri/matrix_multiplication`

October 2026

Abstract

This experiment examines how matrix multiplication performance can be improved through dense optimisation techniques and sparse data representation. Three dense implementations were compared: a basic triple-loop algorithm, loop reordering and cache blocking. CSR multiplication was also evaluated using matrices with different sizes and proportions of zero entries.

For the largest dense matrices, loop reordering reduced execution time by approximately 26% compared with the baseline, while blocking produced a smaller improvement. CSR achieved much larger speedups at high sparsity levels, although conversion time and memory requirements remained relevant. The results show that the effectiveness of each approach depends on the memory access pattern and the structure of the input data.

1 Introduction

Matrix multiplication is an important operation in many computational applications, including machine learning and large-scale data processing. However, its performance can vary considerably depending on how the algorithm accesses memory and how the matrices are represented.

In Assignment 1, basic matrix multiplication was implemented in Python, Java and C. The experiments showed differences in execution time and memory usage across the three implementations. Building on that work, this assignment focuses on improving the performance of matrix multiplication using C.

Two aspects were investigated. First, the effect of loop reordering and cache blocking was examined using dense matrices. Second, multiplication using the Compressed Sparse Row (CSR) representation was compared with a dense method at different sparsity levels.

The main objective was to determine under which tested conditions these approaches improve execution time and memory efficiency. All methods were evaluated using controlled inputs, repeated measurements and correctness checks.

More specifically, the experiment addressed two questions: how do loop reordering and cache blocking affect dense matrix multiplication performance, and under which tested sparsity levels does CSR provide an advantage over dense multiplication?

2 Background

Matrix multiplication performance depends not only on the number of arithmetic operations, but also on how data is accessed in memory. Since matrices in C are stored in row-major order, accessing consecutive elements of the same row generally provides better memory locality than repeatedly moving between different rows.

Loop reordering can improve this access pattern without changing the theoretical complexity of multiplication. Cache blocking follows a related idea, dividing the computation into smaller regions so that data can be reused more effectively. However, the benefit of these techniques depends on factors such as matrix size, block size and hardware characteristics [3, 4].

Sparse representations address a different problem. Instead of improving access to all matrix elements, they avoid storing and processing many of the zero entries. Their usefulness therefore depends on the sparsity of the inputs and the additional costs associated with the representation.

3 Methodology

3.1 Experimental Environment

All experiments were conducted on a Windows computer equipped with an Intel Core Ultra 7 155H processor, 16 physical cores, 22 logical processors and 32 GiB of RAM.

The implementations were written in C and compiled through MSYS2 UCRT64 using GCC 16.2.0 with the `-O2` and `-std=c11` flags. All multiplication algorithms were executed sequentially.

The experiments were carried out on Windows 11 Home. Python 3.12 was used to generate the input matrices, measure process memory and produce the figures. The main Python libraries used for the analysis were Matplotlib 3.10.8 and psutil 7.1.0.

3.2 Dense Matrix Implementations

Three dense matrix multiplication methods were implemented in C. All of them used the same input matrices and integer data type, allowing their execution times to be compared under equivalent conditions.

The first method was the basic triple-loop algorithm from Assignment 1, which was used as the baseline. It follows the i-j-k loop order, computing each element of the result by multiplying a row of A with a column of B.

The second method applied loop reordering, changing the loop order to i-k-j. This allows consecutive elements of B and C to be accessed within the innermost loop, improving memory locality. Although the number of arithmetic operations remains unchanged, the new access pattern was expected to make better use of the cache.

The third method used cache blocking with a fixed block size of 32. Instead of processing the matrices as a whole, multiplication was divided into smaller regions to encourage data reuse. The blocks at the matrix boundaries were adjusted when the dimensions were not multiples of 32.

All three methods retained the same cubic time complexity, $O(n^3)$, and were executed sequentially.

3.3 Sparse Matrix Implementation

For the second part, matrix multiplication was implemented using the Compressed Sparse Row (CSR) representation [1].

Unlike dense storage, CSR keeps only nonzero values, together with their column indices and an array indicating where each row begins. This reduces the amount of data stored when a matrix contains a large proportion of zeros.

The input matrices were initially generated in dense format and then converted to CSR. During multiplication, the implementation iterated over the nonzero elements of A and the corresponding nonzero elements of B, avoiding operations involving zero entries.

The resulting matrix was stored in dense format. This simplified correctness checks and allowed a direct comparison with the dense multiplication method, although it also introduced an $O(n^2)$ storage requirement for the output.

CSR multiplication was compared with the i-k-j dense implementation from Part A. The conversion time was recorded separately to distinguish the cost of preparing the sparse representation from the multiplication itself.

3.4 Input Data and Correctness

For the dense experiments, eight square matrix sizes were tested: 10, 50, 100, 200, 300, 500, 750 and 1000. The smaller input matrices were reused from Assignment 1, while the larger ones were generated using the same procedure.

All matrices contained integer values between 0 and 9. All implementations used 32-bit `int` values. Given the tested sizes and value ranges, no integer overflow was expected. The resulting values remained well below the maximum value of a 32-bit signed integer. Python's random generator was used with fixed seeds of 42 and 43 for matrices A and B, respectively [2]. The inputs were saved as CSV files so that every implementation could use the same data.

For the sparse experiments, five sizes were selected: 100, 300, 500, 750 and 1000. Four sparsity levels were considered, corresponding to 50%, 90%, 95% and 99% zero entries. Nonzero positions were selected randomly without replacement, with values between 1 and 9 and the same fixed seeds.

Correctness was checked before analysing performance. The dense implementations were tested using small matrices with known results, identity matrices and zero matrices. Their outputs were also compared with reference results generated in Python.

The CSR implementation was tested with matrices containing empty rows and with multiplication by a zero matrix. For every experimental configuration, its result was compared element by element with the dense multiplication result. All tested configurations passed validation.

3.5 Benchmarking Procedure

Execution time was measured using `clock_gettime(CLOCK_MONOTONIC)`. Input loading and matrix generation were performed outside the timed region. The multiplication measurements included output initialisation, while correctness checks were excluded.

Each configuration included one warm-up execution followed by five measured repetitions. In the dense experiment, batching was applied to the smallest size ($n = 10$), using 1000 multiplications per measurement. In the sparse experiment, 10 multiplications were performed per measurement for $n = 100$. In both cases, elapsed time was divided by the batch size. The execution order of dense and CSR multiplication was alternated between repetitions.

The median was used to represent execution time, while the interquartile range (IQR) was used to describe variability. All individual measurements were saved in CSV files for later analysis.

For CSR, conversion time was measured separately and included converting both input matrices, A and B, from dense storage to CSR. Conversion measurements were not batched in the same way as the smallest multiplication configurations.

Memory usage was evaluated using two metrics. Array storage was estimated from the dimensions and data structures used by each method. Peak process Resident Set Size (RSS) was also measured using Python's `psutil` library.

Each memory configuration was executed once using a separate memory mode, without modifying the timing results. In this mode, the programs load the same input matrices and perform multiplication without recording benchmark times or writing CSV files. The dense implementations allocate matrices A, B and C, while the sparse implementation also allocates CSR representations of both inputs and a dense reference result. Process memory was sampled approximately every 1 ms. These measurements include program overhead and temporary allocations, and may not capture the exact peak between samples.

The largest tested matrix size was 1000×1000. For the largest configurations, individual multiplication times remained below one second, while the highest observed process RSS was approximately 30.47 MiB. These were observed limits rather than a predefined resource budget. Larger matrices were not tested, so the maximum feasible size was not determined.

4 Results

4.1 Dense Matrix Optimisation

Table 1 presents the median execution times for the three dense multiplication methods. As expected, execution time increased considerably with matrix size, although the differences between methods became more noticeable for larger inputs.

Table 1: Median execution time for dense multiplication (ms).

Size	Baseline	Reordered	Blocked
10	0.0007	0.0006	0.0006
50	0.0904	0.0540	0.0570
100	0.4396	0.4623	0.4343
200	4.0145	3.1326	3.9979
300	9.9929	10.4805	11.1471
500	50.0046	45.5422	52.9480
750	213.9031	155.8522	180.6977
1000	500.0392	369.4311	430.0806

Figure 1 shows the execution time trends. Loop reordering performed particularly well for the largest matrices, while cache blocking also reduced execution time compared with the baseline at sizes 750 and 1000.

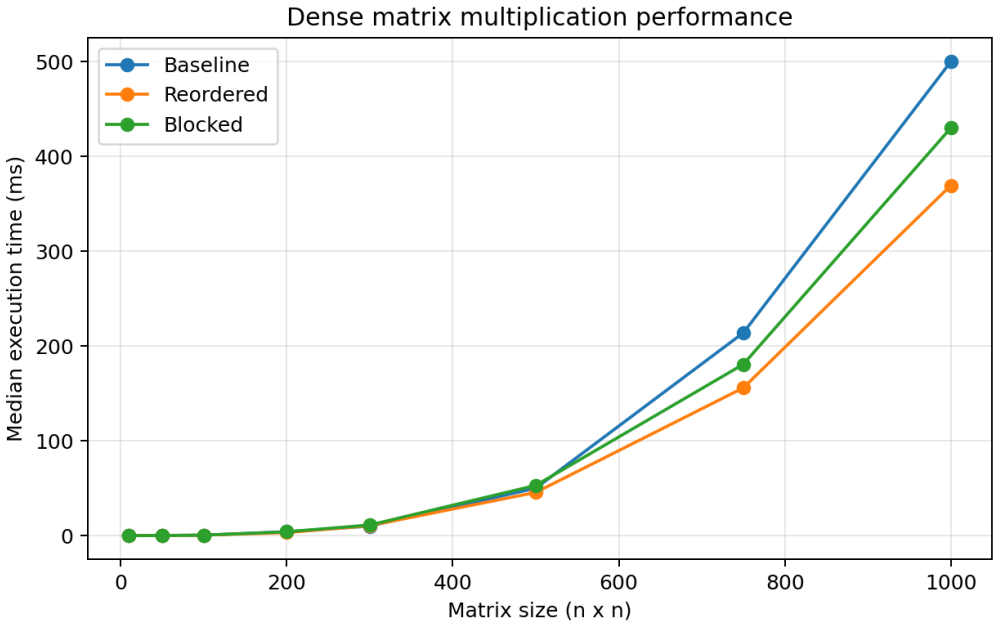


Figure 1: Median execution time for the three dense multiplication methods.

To compare the optimisations more directly, speedup was calculated as the baseline execution time divided by the optimised execution time.

$$S(n) = \frac{T_{\text{baseline}}(n)}{T_{\text{optimised}}(n)}$$

As shown in Figure 2, loop reordering achieved a speedup of approximately 1.35 for matrices of size 1000, corresponding to a reduction in execution time of around 26%. Cache blocking reached a speedup of approximately 1.16 for the same size.

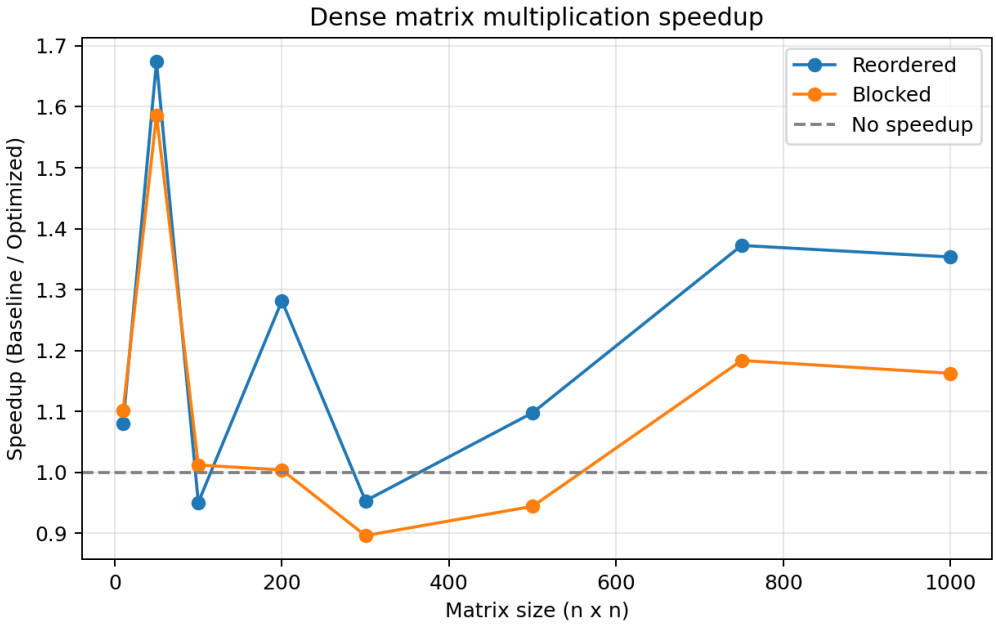


Figure 2: Speedup of the optimised dense methods relative to the baseline.

The smaller configurations did not show a consistent advantage for either optimisation. Their execution times were relatively short, and some measurements showed noticeable variability. For example, the baseline had an IQR of 0.0599 ms at size 50, compared with a median of 0.0904 ms.

Memory consumption followed a similar pattern for all three implementations, as shown in Figure 3. Peak process RSS increased with matrix size, but changing the loop order or introducing blocking did not result in substantial differences in observed memory usage.

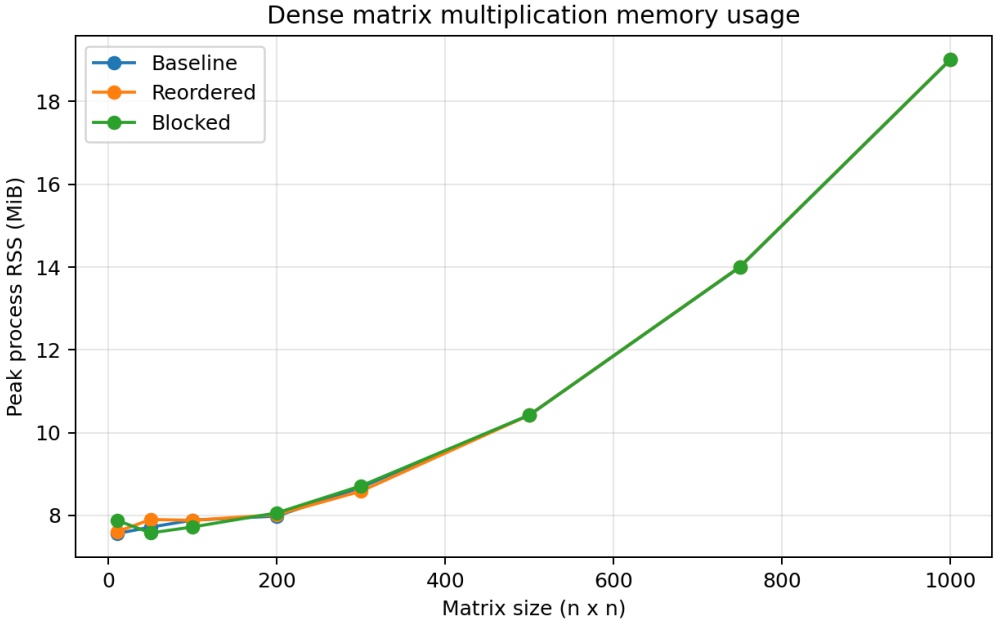


Figure 3: Peak process RSS for the dense multiplication methods.

Overall, loop reordering provided the most consistent performance improvement for larger

dense matrices in this experiment. Cache blocking also offered benefits at some sizes, although its performance was less consistent across the tested configurations.

4.2 Sparse Matrix Multiplication

The second experiment examined how sparsity affects matrix multiplication performance. CSR multiplication was compared with the dense i-k-j implementation using the same input matrices. The results showed a clear difference between the two methods, particularly at higher sparsity levels.

Table 2 presents the median execution times for matrices of size 1000. While dense multiplication remained relatively similar across the tested sparsity levels, CSR became considerably faster as the number of nonzero elements decreased.

Table 2: Median execution times for 1000 x 1000 matrices (ms).

Zeros (%)	Dense	CSR	Conversion
50	346.13	111.84	8.01
90	348.18	5.81	3.59
95	352.63	2.05	2.86
99	395.12	0.46	1.85

Table 3 shows the number of nonzero elements in the input and output matrices for the largest tested size. Although the number of nonzero elements in A and B decreased with sparsity, the result was often considerably denser than either input.

Table 3: Nonzero elements for 1000 x 1000 matrices.

Zeros (%)	NNZ A	NNZ B	NNZ C
50	500000	500000	1000000
90	100000	100000	999954
95	50000	50000	918015
99	10000	10000	95021

The difference can also be observed in Figure 4, which compares both methods for matrices with 90% zero entries. Dense execution time increased rapidly with matrix size, whereas CSR multiplication remained much faster throughout the tested range.

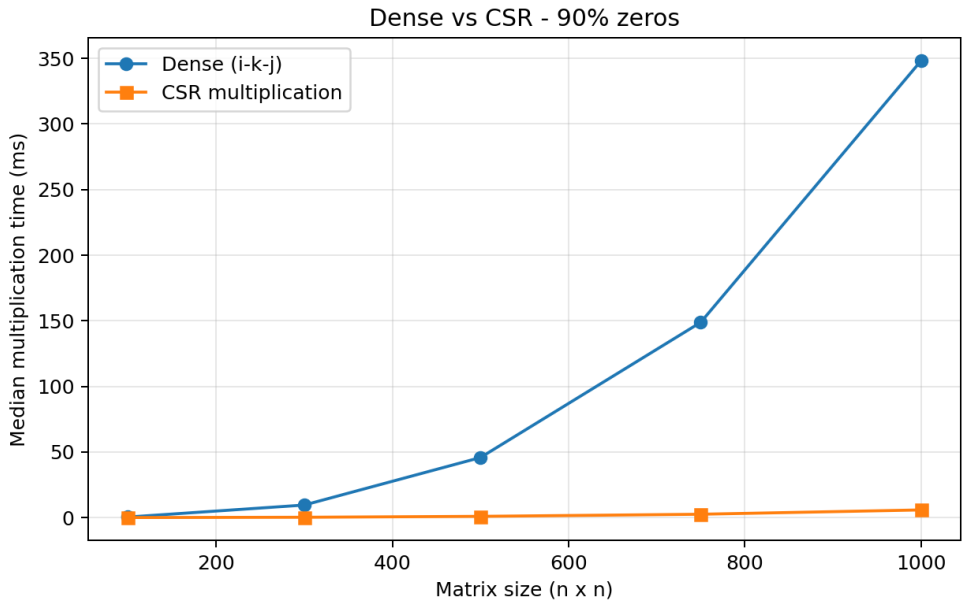


Figure 4: Dense and CSR multiplication times with 90% zero entries.

Figure 5 shows the speedup obtained with CSR at each sparsity level. The advantage generally increased as the proportion of zero entries grew. For matrices of size 1000 with 99% zeros, CSR multiplication was several hundred times faster than the dense method. The speedup calculated from the measured medians was approximately 855, although this configuration also showed considerable timing variability.

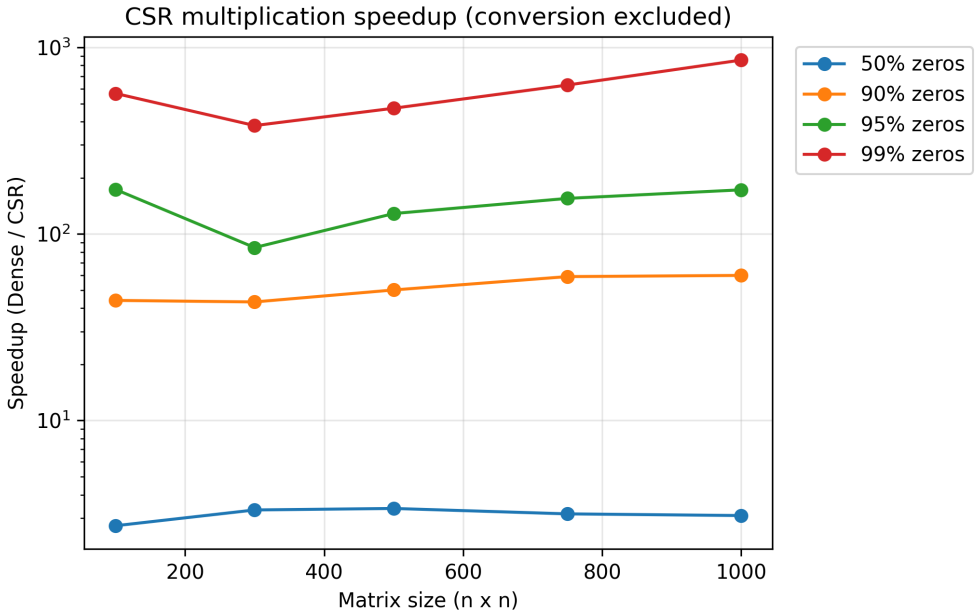


Figure 5: CSR multiplication speedup relative to the dense method, excluding conversion time. The vertical axis uses a logarithmic scale.

These speedups refer only to multiplication. Conversion was measured separately because the input matrices were originally stored in dense format. Although conversion introduced an additional cost, the combined times remained below the dense multiplication times for the configurations shown in Table 2.

4.2.1 Memory Usage

Memory requirements also changed with sparsity. Figure 6 compares the estimated storage of three dense matrices with that of two CSR inputs and a dense output, for matrices of size 1000.

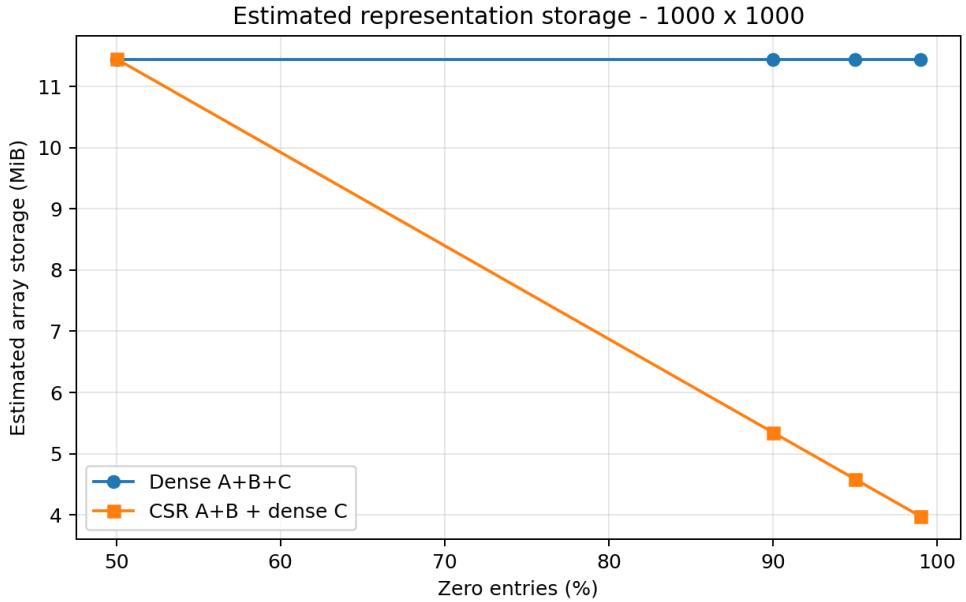


Figure 6: Estimated representation storage at different sparsity levels for matrices of size 1000.

With 50% zero entries, CSR required slightly more storage than the dense representation because of the additional indices. At 99% zeros, however, CSR required approximately 3.97 MiB, compared with 11.44 MiB for dense storage.

A similar trend appeared in the process memory measurements. For size 1000, observed peak RSS decreased from approximately 30.47 MiB at 50% sparsity to 22.99 MiB at 99%. These measurements include the dense input arrays, the reference result and other allocations, so they do not represent the memory used exclusively by CSR.

As shown in Table 3, multiplication often produced a result with many more nonzero elements than the input matrices. This helps explain the choice of a dense output representation and shows that sparse inputs do not necessarily produce a sparse result.

Overall, CSR offered the greatest benefits when the proportion of zero entries was high. Its effectiveness depended on both the reduced multiplication work and the additional storage and conversion costs associated with the representation.

5 Discussion

The two experiments showed that matrix multiplication performance can be improved in different ways, either by changing how data is accessed or by reducing the number of operations performed.

In the dense experiment, loop reordering generally provided better results for larger matrices. This is consistent with the improved memory access pattern of the i-k-j loop order, which allows consecutive elements of B and C to be processed. Although the algorithm still performs $O(n^3)$ arithmetic work, accessing data more efficiently can reduce execution time.

Cache blocking also improved performance for some of the larger configurations, but its advantage was less consistent. One possible explanation is that the fixed block size of 32 was not equally suitable for every matrix size. The additional loop structure and compiler optimisations

may also have influenced the measurements. Since other block sizes were not tested, the results do not establish whether 32 was the most effective choice.

The sparse experiment showed a much stronger relationship between input structure and execution time. Unlike the dense method, CSR multiplication only processes nonzero elements and their corresponding contributions. As sparsity increased, the amount of work was reduced considerably, which explains the large speedups observed in the most sparse configurations.

However, these advantages involve additional costs. CSR requires column indices and row pointers, while converting dense inputs introduces extra processing time. At lower sparsity levels, these costs become more relevant, particularly when considering memory usage. The dense output also remains a limitation, since its storage requirement does not decrease with input sparsity.

Several factors should be considered when interpreting the results. All experiments were performed on a single computer using one compiler configuration, and the implementations were sequential. The sparse matrices were generated with uniformly distributed nonzero positions, so other distributions could produce different results. Furthermore, process memory was measured through periodic sampling, which may not capture the exact peak usage.

Variability between separate executions should also be considered. The five repetitions were performed consecutively within the same process, so the reported IQR mainly reflects short-term variation rather than differences between independent runs. The processor also has a hybrid architecture, and no explicit CPU affinity was set. Therefore, moderate performance differences, particularly for small matrices or the blocked implementation, should be interpreted with caution.

The execution times also showed some variability, especially for smaller configurations. Although repeated measurements and warm-up runs were used, operating system activity, caching and compiler behaviour could still influence the results. Therefore, the conclusions are limited to the tested implementations, datasets and execution environment.

Overall, the results suggest that improving memory locality is useful for dense multiplication, while sparse representations can provide substantially greater benefits when the input matrices contain a high proportion of zeros. Neither approach guarantees the same improvement under every configuration.

6 Conclusion

This assignment investigated how dense matrix multiplication can be optimised and under which sparsity conditions CSR offers an advantage over a conventional dense implementation.

For dense multiplication, loop reordering achieved the best results for larger matrices, reducing execution time by approximately 26% for the largest tested size. Cache blocking also improved performance in some configurations, although its results were less consistent. Neither optimisation significantly changed memory usage.

The sparse experiment showed that CSR can provide much greater performance improvements when most input elements are zero. Both execution time and estimated storage decreased as sparsity increased, although conversion costs and the dense output representation remained important considerations.

These results show that matrix multiplication performance depends on memory access patterns and data representation, not just the number of operations. Further experiments could examine different block sizes and sparse matrix structures.

Use of Artificial Intelligence

ChatGPT (OpenAI) was used to assist with code development, debugging and report writing. Claude (Anthropic) was also used to review the work and suggest improvements.

All experiments were executed and validated locally. The measurements were obtained from the actual runs, and the author was responsible for reviewing the results and interpreting the findings.

References

[1] Netlib, *Templates for the Solution of Linear Systems: Building Blocks for Iterative Methods*, section on sparse matrix storage formats. Available: `https://netlib.org/linalg/html_templates/node91.html`

[2] Python Software Foundation, *random — Generate pseudo-random numbers*. Available: `https://docs.python.org/3/library/random.html`

[3] Intel, *Optimize Memory Access Patterns using Loop Interchange and Cache Blocking Techniques*, Intel Advisor Cookbook, 2024. Available: `https://www.intel.com/content/www/us/en/docs/advisor/cookbook/2024-2/optimize-memory-access-patterns.html`

[4] Cornell University, *Cache Blocking & Optimization*, CS 3410, Spring 2025. Available: `https://www.cs.cornell.edu/courses/cs3410/2025sp/assignments/cacheblock/instructions.html`